객체지향 프로그래밍 고급

3강 : 생성자와 소멸자

생성자와 소멸자

- 생성자는 객체를 생성할 때 자동적으로 호출되는 함수
- 소멸자는 객체가 소멸될 때 자동으로 호출되는 함수

```
class AAA
{
public:
    AAA() { }
    ~AAA() { }
};
```

```
class Cat
{
public:
    Cat();
    ~Cat();
};
Cat::Cat() {
 . . .
}
Cat::~Cat() {
 . . .
}
```

생성자

- 특징
  - 클래스의 이름과 함수의 이름이 동일하다.
  - 반환형이 없다.
  - 생성자는 한 클래스에서 여러 개 만들 수 있다. 단, 매개변수 개수나 타입이 다르게 선언되어야 한다.
  - 객체를 생성할 때 딱 한번 호출된다.
- 왜 필요한가?
  - 객체를 생성할 때 해주어야 하는 일들을 한다.('준비')
  - **멤버 변수의 초기화**
  - **객체가 사용할 메모리 준비(동적 메모리 할당)**

기본 생성자

- 생성자가 하나도 없는 클래스는 컴파일러가 자동으로 다음과 같이 기본 생성자를 생성한다.

클래스명 { }

- 멤버들이 초기화되지 않고 객체가 생성된다.

```
Point::Point() { }
```

기본 생성자

```
class Point
{
    private:
        int x, y;
    public:
        Point();   // 객체 생성 시점 아님
        void Show();
} ;
Point::Point()
{
    cout << "생성자 호출"<< endl;
}
void Point::Show()
{
    cout << "데이터 출력" << endl;
}
```

```
int main()
{
    cout << "main 시작<<endl;
    Point pt;      // 객체 생성 시점
    pt.Show();
    return 0;
}
```

기본 생성자

- 생성자 내에서 멤버변수를 초기화

```
class Dog
{
    private:
        int age;
        double weight;
    public:
        Dog();
        void Show();
} ;

Dog::Dog()
{
    age = 3;
    weight = 3.4;
};
```

```
void Dog::Show()
{
    cout << "나이: " << age << endl;
    cout << "몸무게: " << weight << endl;
}

int main()
{
    Dog happy;  // 객체가 생성되는 시점
    happy.Show();
    return 0;
}
```

매개변수있는 생성자

- 멤버변수가 초기값을 갖도록 매개변수있는 생성자를 사용한다.

```
클래스명::클래스명(매개변수리스트)
{
    멤버변수를 초기화하는 코드;
}
```

매개변수있는 생성자

```
class Dog
{
private:
    int age;
    double weight;
public:
    Dog(int a, double w);
    void Show();
} ;

Dog::Dog(int a, double w)
{
    age = a;
    weight = w;
};
```

```
void Dog::Show()
{
   cout << "나이: " << age << endl;
   cout << "몸무게: " << weight
<< endl;
};

int main()
{
   Dog happy(3, 4.3);
   happy.Show();
   return 0;
}
```

매개변수있는 생성자

- 객체가 많아지면 배열을 사용하는 것이 효과적이다.

```
class Dog
{
private:
      int age;
      double weight;
public:
      Dog(int a, double w);
      void Show();
} ;

Dog::Dog(int a, double w)
{
      age = a;
      weight = w;
};
void Dog::Show()
{
    cout << “나이: “ << age << endl;
    cout << “몸무게: “ << weight << endl;
};
```

```
int main()
{
    Dog d[ ] = { Dog(3,4), Dog(1,2),
Dog(5,6) };

    for (int i=0; I,3; i++)
          d[i].show();

    return 0;
}
```

소멸자

- 특징
  - 객체값을 소멸시키고 할당한 메모리를 해제한다.
  - 정적 생성의 경우, 객체를 정의한 멤버 함수가 끝날 때 실행된다.
  - 동적 생성의 경우 delete 연산자로 기억장소를 해제할 때 실행된다.
  - 소멸자의 실행 순서는 객체 생성 순서와 반대이다.
- 왜 필요한가
  - 소멸자는 '정리'에 해당하는 작업
  - **동적 메모리 해제**
  - **파일 close**

소멸자

- 객체를 더 이상 사용하지 않을 때 객체를 제거하여 작업을 정리
(객체값을 소멸시키고 할당한 메모리도 해제)
- 객체가 유효 범위를 벗어날 때 자동으로 호출

```
Dog::~Dog()
{
}
```

소멸자

```
class Dog
{
private:
    int age;
    double weight;
public:
    Dog();
    ~Dog();
    void Show();
} ;

Dog::Dog() {
    cout << "생성자 호출\n";
    age = 3;
    weight = 4.3;
}
```

```
Dog::~Dog() {
    cout << "소멸자 호출 \n";
}

void Dog::Show()
{
   cout << "나이: " << age << endl;
   cout << "몸무게: " << weight << endl;
}

int main()
{
   Dog happy;
   happy.Show();
   return 0;
}
```

[예제2] 매개변수있는 생성자

다음과 같은 속성을 갖는 Student 클래스를 설계하고 아래 데이터를 출력하시오.

속성	데이터타입	데이터
학번	string	0821001
이름	string	박달재
객체지향점수	int	95
Unity 점수	int	88

[예제3] 매개변수있는 생성자

```
class Student
{
private:
    string id;
    string name;
    int   Kor;
    int   Eng;
public:
    Student();
    ~Student();
    void Show();
};
```

초기화 데이터

학번 : “1234567”

이름 : “홍길동”

Kor: 98;

Eng: 100;

[예제3]스피드 구구단 클래스

30초동안 구구단을 맞추는 게임을 클래스로 작성하라.

! 오버로드란?

- **함수 이름은 동일하지만 매개변수 자료형이나 매개변수 개수가 다른 경우를 오버로드(overload)라고 함**

```
int square(int x);
double square(double d);

int main()
{
    int ir = square(8);
    double dr = square(6.5);

    return 0;
}
```

```
int square(int x)
{
    return x * x;
}

double square(double d)
{
    return d * d;
}
```

생성자의 오버로드

- 하나의 클래스에 두 개 이상의 생성자가 존재
- 생성자 이름은 동일하고, 매개변수의 개수나 매개변수의 자료형이 다르게 선언되어야 함

생성자의 오버로드

```
class Dog
{
private:
    int age;
    double weight;
public:
    Dog();
    Dog(int a, double w);
    void Show();
};
Dog::Dog()
{
    age = 3;
    weight = 3.4;
}
Dog::Dog(int a, double w)
{
    age = a;
    weight = w;
}
```

```
void Dog::Show()
{
    cout << "나이: " << age << endl;
    cout << "몸무게: " << weight << endl;
}

int main()
{
    Dog happy;
    Dog marry(5, 7.9);

    happy.Show();
    marry.Show();

    return 0;
}
```

[예제4]

문제: Rectangle 클래스를 작성하시오.

멤버 변수

- width
- height

생성자

- 기본 생성자 → width=1, height=1
- 생성자(int w) → width=w, height=1
- 생성자(int w, int h) → width=w, height=h

멤버 함수

- Area() : 넓이 출력

[예제5]자동차 스피드 게임

게임 규칙

- 두 대의 자동차가 게임을 한다.
- 자동차마다 최고속도가 다르다.
  - 기본생성자와 매개변수있는 생성자가 있다. (차의 이름과 최고 속도)
- 5번 가속한다.
- 매번 `10~30km/h` 랜덤으로 속도가 증가한다.
- 최고 속도를 넘으면 최고 속도로 유지한다.
- 먼저 최고 속도에 도달한 자동차가 승리한다.

```
int main()
{
    srand((unsigned)time(0));

    Car car1("Ferrari", 220);
    Car car2("Porsche", 200);

    cout << "===== 자동차 스피드 게임 =====\n\n";

    for (int i = 1; i <= 5; i++)
    {
        cout << "------ " << i << "번째 가속 ------" << endl;

        car1.Accelerate();
        car2.Accelerate();

        car1.Print();
        car2.Print();

        if (car1.Finish())
        {
            cout << "\n우승 : " << car1.GetName() << endl;
            break;
        }

        if (car2.Finish())
        {
            cout << "\n우승 : " << car2.GetName() << endl;
            break;
        }

        cout << endl;
    }
}
```

[예제5-2] 스피드게임 업그레이드

- **브레이크 기능 추가** `Brake()`를 만들어 속도를 20 감소시킨다.
- **부스터 기능 추가** `Boost()`를 만들고 속도를 50 증가시킨다(최고속도는 넘지 않음).
- **자동차 3대로 경기하기** BMW, Ferrari, Porsche를 생성하여 가장 먼저 최고속도에 도달한 자동차를 출력한다.
- **연료(Fuel) 추가** 가속할 때마다 연료가 10씩 감소하고, 연료가 0이면 더 이상 가속하지 못한다.

생성자와 소멸자

const 객체

const 객체

- 변수, 함수, 객체에 const 키워드를 붙여서 상수로 선언할 수 있다.
- const 객체를 사용하는 목적은 객체의 값이 변경되는 것을 금지하여 보호하기 위함이다.

```
const int num = 10;    // 변수를 상수화
```

- const로 선언된 변수는 상수로 인식되므로 변수를 선언할 때 반드시 초기화시켜야 한다.

const 멤버변수

```
class PhoneBook
{
private:
    const int id;
    string name;
    string phoneNo;
public:
    PhoneBook(int d, string n, string pno);
    void Show();
};
```

```
PhoneBook::PhoneBook(int d,string n,string pno) :
id(d)
{
    name = n;
    phoneNo = pno;
}
void PhoneBook::Show()
{
    cout << id << " " << name << " " << phoneNo << endl;
}

int main()
{
      PhoneBook b(1, "유준상", "010-111-2222");
      b.Show();
      return 0;
}
```

생성자():id(d)의 의미

- id(d)는 생성자가 호출되기 이전에 id=d문을 실행(initializer)하라는 의미
- 만일 initializer가 없다면 생성자가 호출되면서 id가 쓰레기값으로 초기화된 후 변경할 수 없게 됨

const 멤버함수

- const로 선언된 멤버함수는 상수타입으로 선언된 것이다.
- const 멤버함수는 멤버변수의 값을 변경하지 않는다.
- const 함수 내에서는 const 함수만 호출할 수 있다.

void 함수명(매개변수) const

{ . . .

}

const 멤버함수(잘못된 부분을 찾으시오.)

```
class SimpleClass
{
    private:
            int num;
    public:
            void InitNum(int n)
            {
                num = n;
            }
            int GetNum()
            {
                return num;
            }
            void ShowNum()  const
            {
                 cout << GetNum() << endl;
            }
};
```

const함수내에서는 const가 아닌 함수의 호출이 제한된다.

const 객체

- 이 객체의 데이터 변경을 허용하지 않음을 의미
- const 객체를 대상으로는 const 멤버함수만 호출이 가능

```
class Simple
{
   . . .
};

int main()
{
        const Simple sim(10);
        . .
}
```

main()함수에서 잘못 불려진 멤버함수는 무엇인가?

```
class SoSimple
{
	int num;
public:
	SoSimple(int n) : num(n)
	{}
	SoSimple AddNum(int n)
	{
		num += n;
		return *this;
	}
	void SimpleFunc()
	{
		cout << "SimpleFunc:" << num << endl;
	}
	void SimpleFunc() const
	{
		cout << "const SimpleFunc:" << num << endl;
	}
};
```

```
int main()
{
	SoSimple obj1(2);
	const SoSimple obj2(7);

	obj1.SimpleFunc();
	obj2.SimpleFunc();

	obj1.AddNum(3);
	obj2.AddNum(3);

	obj1.SimpleFunc();
	obj2.SimpleFunc();

	return 0;
}
```

[예제]Rectangle

- xy그래프에서 사각형은 두 개의 x, y 좌표를 이용하여 표현할 수 있다.

  ③ Rectangle 클래스의 속성인 x, y 좌표값을 Point 클래스의 두 개의 객체로 대체하라.

  좌표값 (10, 10), (50, 100)을 갖는 사각형의 면적을 구하고 아래와 같이 출력하라.

  좌 상단 : (10, 10)

  우 하단 : (50, 100)

  면적 : 3600